Unidad 4
Actividad Práctica

PARTE-2: Equipo EduControl — Benitez, Danilo Agustín · Gomez, Javier

Esta parte será coevaluada en la Segunda Instancia de evaluación el día 27 de octubre.

1. Seleccionar una funcionalidad que funcione y esté completa.
   Inscripción a deportes con control de cupo y de horarios (RF-09, HU6). El tutor inscribe a su hijo desde la autogestión → el sistema controla que no supere dos deportes (RN-03), que no se superpongan entre sí (RN-04) ni con el horario de cursado (RN-05) → acepta la inscripción o la rechaza explicando el motivo. Cubierta por 16 pruebas automáticas (6 de integración y 10 casos unitarios de superposición horaria) y por la prueba end-to-end del perfil Tutor.
2. De acuerdo al análisis del contenido de la unidad 4, completar la siguiente tabla: *(*) aplicado en esta entrega, ver punto 3.b*

Contenido	**Aplicado**		**N/A**
	Ejemplificar con las herramientas y/o estrategias aplicadas según corresponda	Ejemplificar un flujo	
Integración Continua	GitHub Actions: motor de CI (`.github/workflows/ci.yml`) que se ejecuta ante cada push y pull request, con una rama por integrante y PR hacia `main`.	1. El integrante sube cambios a su rama (ej. `Nilo`) o abre un PR hacia `main`. 2. El evento dispara `ci.yml` en la nube. 3. El runner instala dependencias, las audita, revisa el estilo y corre los 56 tests con cobertura; deja ✓ o ✗ en el commit y en el PR.	
Despliegue Continuo			No aplica
Build (*)	`composer validate` + `composer install` con `composer.lock` (versiones exactas) y caché de Composer por hash del lock; PHP 8.3 con `setup-php`.	Push → se restaura la caché → `composer validate` comprueba que el lock coincida con `composer.json` → `composer install`. En `main`, `git archive` + `composer --no-dev` arman `educontrol-<commit>.zip`.	
Pipelines (*)	GitHub Actions: un workflow con cuatro trabajos encadenados con `needs`: pruebas → paquete → verificar-producción y notificar. `concurrency` cancela corridas viejas.	Commit → Build → Seguridad (`composer audit`) → Lint (Pint) → Test (PHPUnit + PCOV) → Package → Verify → Notify → emite resultado ✓/✗.	
Variables de Entorno	`.env` con su `.env.example` (ignorado por git), `phpunit.xml` con SQLite en memoria, variable de repositorio `URL_PRODUCCION` y `GITHUB_TOKEN` con permisos mínimos.	En local Laravel lee el `.env`; en el CI el workflow crea su propio `.env` con una `APP_KEY` nueva, sin credenciales reales. Sin `URL_PRODUCCION` el job de producción se omite.	
Gestión de Artefactos	`actions/upload-artifact`: informe de tests y cobertura (`junit.xml`, `clover.xml`, HTML) por 14 días y paquete `.zip` desplegable por 30 días.	Merge a `main` → el job paquete arma el `.zip` sin tests ni `.env` → Verify revisa su contenido → queda descargable desde la pestaña Actions.	
Integración de repositorios con herramientas CI/CD (*)	Integración nativa GitHub ↔ GitHub Actions, insignia del CI en el README y Dependabot.	Cada push y PR dispara `ci.yml`. Cada lunes Dependabot abre PR con las actualizaciones, que pasan por el mismo CI (#65, #66 y #67 en verde).	
Pull Requests y Merge Requests	PR hacia `main` con plantilla (checklist del autor y de quien revisa) y `CODEOWNERS`, que pide la revisión al otro integrante.	Rama → PR → CI en verde → revisión del otro integrante → *Squash and merge* → `Closes #N` cierra el issue.	
Issues (*)	GitHub Issues: 59 issues con etiquetas por integrante y criterios de aceptación; plantillas *Tarea* y *No conformidad*.	Creación del issue (ej. RF-09, inscripción a deportes) → asignación a un integrante → PR con `Closes #N`. Si falla `main`, el CI abre solo un issue `no-conformidad`.	
Wikis	Reemplazada por la carpeta `docs/` versionada (la wiki de GitHub no está disponible en el plan del repositorio).		No aplica
Tablero Kanban	GitHub Projects: columnas Por hacer, En progreso, En revisión y Hecho.	Issue en Por hacer → En progreso al crear la rama → En revisión al abrir el PR → Hecho al mergear en `main`.	
Otro/s (*)	`composer audit`, Dependabot, Laravel Pint, cobertura con PCOV, pruebas end-to-end con Playwright y registro de no conformidades (`docs/calidad.md`).	`composer audit` frena el pipeline si una dependencia tiene una vulnerabilidad publicada; Dependabot propone la actualización en un PR que pasa por el CI.	

3. Contenidos NO APLICADOS,
   a. Seleccionar dos opciones que a su criterio deben aplicarse para optimizar el desarrollo de la aplicación.
      Selección: Seguridad de dependencias y Notificación y registro automático de fallos. Los dos contenidos que figuran como no aplicados (Despliegue Continuo y Wikis) dependen de condiciones externas (una cuenta de Railway y el plan de GitHub), así que se eligieron dos mejoras del pipeline que se podían aplicar completas. Motivo: el `composer.lock` estaba desfasado desde el 22/09 y el aviso pasaba en el log del CI sin que nadie lo leyera; y un fallo en `main` solo llegaba por correo al autor, sin quedar registrado.
   b. Realizar la aplicación correspondiente.
      Seguridad de dependencias: `composer validate` en el Build (frena si el lock no coincide), `composer audit` como etapa de Seguridad y Dependabot semanal (Composer con las versiones menores agrupadas, y GitHub Actions). Se corrigió el lock desfasado sin cambiar versiones; resultado: 0 vulnerabilidades, y el 25/10 Dependabot abrió sus primeros 3 PR, los tres en verde.
      Notificación y registro de fallos: el job *notificar* abre un issue con la etiqueta `no-conformidad`, el commit, el autor y el link a la corrida (es el único job con permiso `issues: write`). Se suman plantillas para registrar a mano otras no conformidades y el registro NC-01 a NC-09 en `docs/calidad.md`. Ensayo del 26/10 con `simular_fallo`: el CI abrió solo el issue #68. Esto habilitó además la etapa Verify del paquete y completó Build, Pipelines, Integración de repositorios, Issues y Otro/s.
   c. Entrega: 27 de octubre para su evaluación.

Disclaimer: Despliegue Continuo y Wikis siguen como “No aplica”. El primero ya está preparado (paquete `.zip` verificado, ruta de salud `/up`, job de verificación de producción y guía de Railway con *Wait for CI*) y solo falta crear la cuenta de Railway (issues #6 y #53). La wiki de GitHub requiere cambiar el plan o hacer público el repositorio; mientras tanto, `docs/` cumple la misma función y se revisa en los PR junto con el código.

4. Métricas de productividad
   a. Seleccionar herramientas de métrica que permitan medir el rendimiento de la aplicación con respecto a velocidad de entrega, cuellos de botella y oportunidades de mejora.
      Herramientas a utilizar:
      - GitHub Actions y `gh run list`: duración de cada corrida y tasa de corridas en verde (velocidad de entrega y cuellos de botella).
      - PHPUnit + PCOV: cantidad de tests, test más lento y cobertura de líneas y métodos (calidad y oportunidades de mejora).
      - GitHub CLI (`gh`) y API: tamaño de cada PR, tiempo de abierto a mergeado, revisiones y estado del tablero.
      - `composer audit`: vulnerabilidades de las dependencias.
      - `php artisan reportes:medir`: tiempo de generación de los reportes (RNF-07).
      - Playwright: pruebas end-to-end por rol y de diseño responsivo.
   b. Aplicar las herramientas y mostrar los resultados obtenidos.
      Inicio (sin CI): 2 PR en 2 días; el #59 tenía 153 archivos y +16.517 líneas y se mergeó en 1 min 38 s sin revisión. 0 corridas de CI. 50 tests en 6,4 s; el más lento (exportar reportes a PDF y planilla) concentra el 34 % de la suite. Cobertura sin medir; al medirla en el Pipeline v1: 76,6 % de líneas, con los ABM de configuración entre 8 % y 20 %. Tablero: 49 de 50 issues cerrados fuera de Hecho. Cuello de botella: los issues de despliegue, bloqueados por la cuenta de Railway. Errores ocultos: un warning de PHPUnit que hacía fallar la suite y un `composer.lock` desfasado. Reportes: peor caso de 0,94 s con 3003 alumnos (límite de 3 s).
   c. En función de los resultados obtenidos, especificar estrategias de mejoras,
      - **Verificación:** pipeline de CI en cada push y PR; se corrigió el warning de PHPUnit en la primera corrida.
      - **Calidad:** tests para los ABM de configuración (alta, edición, duplicados y baja): los cuatro controladores pasan del 8–20 % al 100 %.
      - **Lotes grandes:** PR chicos con el CI en verde, `CODEOWNERS` y checklist de revisión.
      - **Seguridad y fallos visibles:** `composer validate`/`audit`, Dependabot y el job Notify (punto 3.b).
      - **Tablero:** se movieron las tarjetas a Hecho y se activó el flujo cerrado → Hecho.
      - **Usabilidad:** 53 pasos end-to-end por rol y en celular; encontraron un defecto (en iPhone la pantalla se agrandaba al tocar el selector de deportes) que se corrigió.

d. Aplicar las estrategias de mejoras y mostrar los resultados obtenidos mediante una tabla comparativa.

Métrica	Inicio (sin CI)	Pipeline v1 (22/10)	Pipeline v2 (mejorado, 25/10)	Variación v1 → v2
Pruebas automáticas	50 (a mano)	50	56	+6
Cobertura de líneas	No medida	76,6 %	83,6 %	+7,0 pp
Cobertura de métodos	No medida	61,3 %	75,7 %	+14,4 pp
Etapas automatizadas	0	4	7	+3
Feedback ante un cambio (rama, promedio de corridas en verde)	Manual, sin registro	31,0 s (8 corridas)	32,0 s (10 corridas)	+1 s con 3 etapas más
Pipeline completo en `main` (con paquete)	—	51 s	57 s (promedio de 2)	+6 s (Verify del paquete)
Vulnerabilidades en dependencias	No medidas	No medidas	0	Control en cada corrida
Errores ocultos detectados por el CI	0	1 (warning de PHPUnit)	2 (+ lock desfasado)	+1
Tamaño del PR	153 archivos, +16.517 líneas (#59)	10 archivos, +434 (#61)	15 archivos, +412 (#64) · 7 archivos, +120 (#69)	Lotes chicos
Fallos registrados como issue	Ninguno	Solo correo	Automático (#68)	—
Pruebas por rol y responsivas	Sin ejecutar	Sin ejecutar	53 de 53 OK	1 defecto corregido
Revisiones registradas en PR	0 de 2	0 de 3	0 de 5	Sin mejora (pendiente)

Duraciones tomadas de `gh run list` (desde que se crea la corrida hasta que termina). La revisión cruzada sigue pendiente aunque `CODEOWNERS` ya la pide: queda registrada como no conformidad NC-01.

e. Fecha de entrega: 27 de octubre.